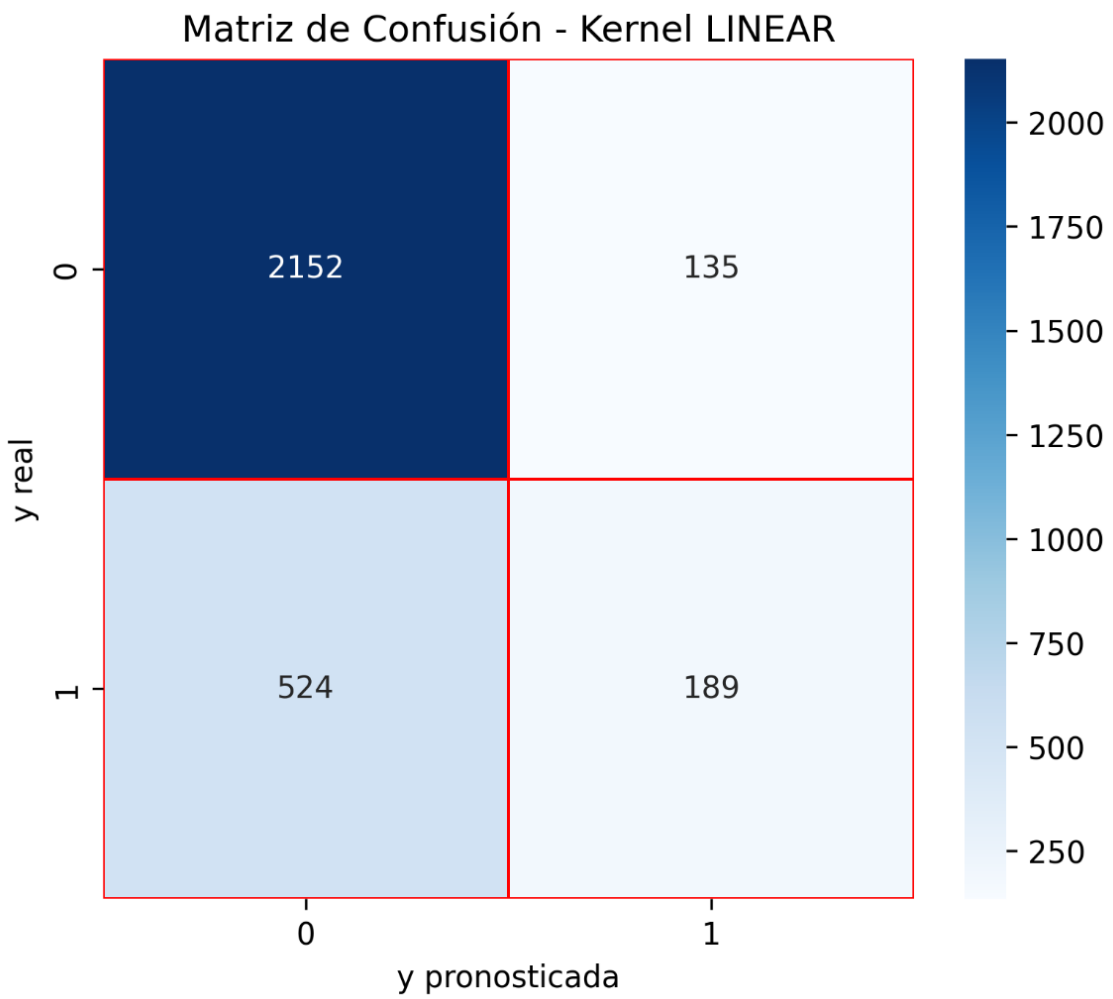
Tarea M22-CD – Yamil Salomón

Introducción y Preprocesamiento de Datos Para comenzar este análisis, abordé el desafío de predecir la retención de nuestros empleados utilizando la base de datos recursos_humanos.csv. Como primer paso fundamental, preparé la información para el algoritmo de Máquinas de Soporte Vectorial (SVM). Como este algoritmo requiere datos numéricos para calcular distancias entre observaciones, utilicé la función pd.get_dummies de Pandas para recodificar las variables de departamento (sales) y categoría de salario (salary) en variables binarias (0 y 1). Posteriormente, realicé un escalado de datos (StandardScaler); esto es crucial con SVM, ya que variables como "horas mensuales" (que superan las 200) y "nivel de satisfacción" (que van de 0 a 1) deben estar niveladas en la misma escala para no sesgar el modelo.

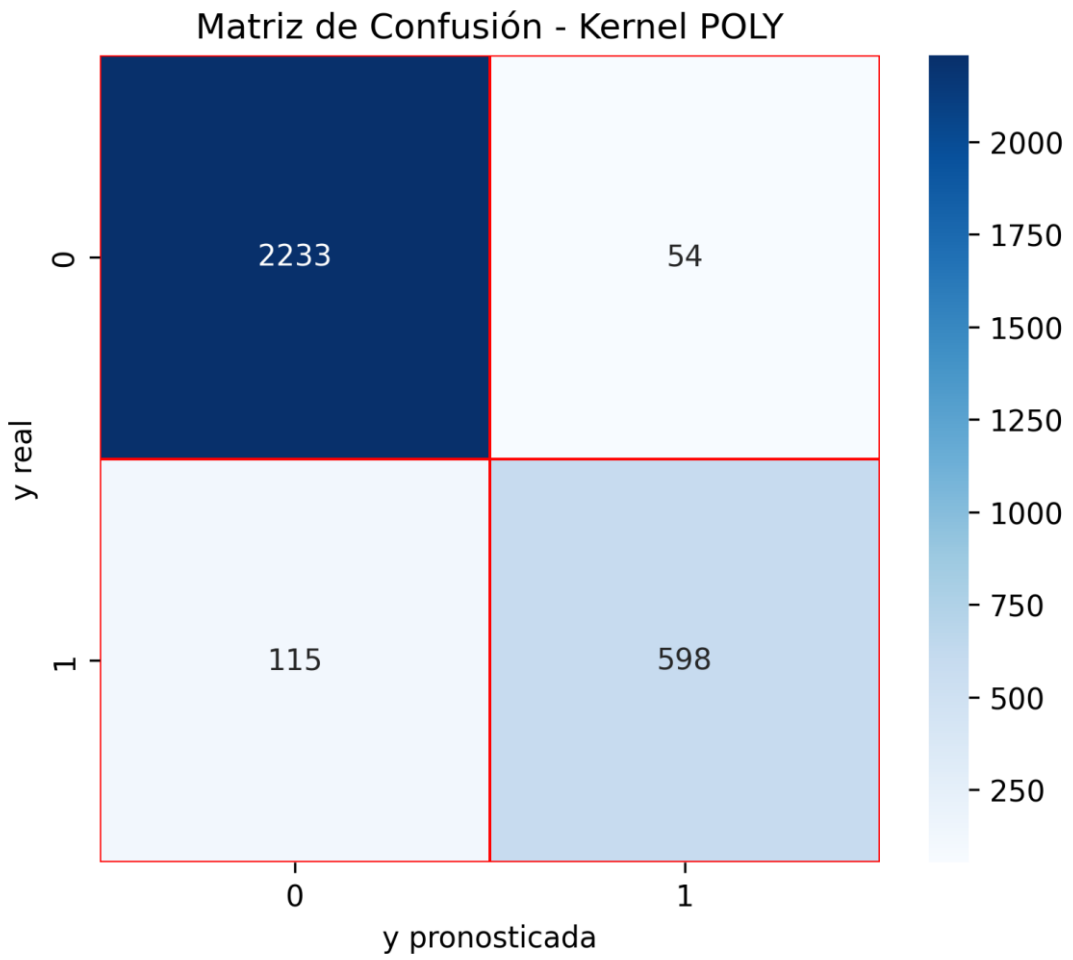
Evaluación e Interpretación de los 4 Kernels Para encontrar la mejor frontera de decisión que lograra separar a los empleados que se quedan de los que se van, puse a competir los cuatro tipos de kernels vistos en clase:

```
import os
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn import svm
from sklearn.metrics import classification_report, confusion_matrix
import warnings
warnings.filterwarnings('ignore')

# 1. Configurar la ruta de trabajo
os.chdir(r"C:\Users\Yamil\OneDrive\Escritorio\Escritorio\Yamil\EBAC (CIENCIA DE DATOS)\Bloque 4 Aprendizaje de máquina supervisado\MOD 4

# 2. Cargar la base de datos
df = pd.read_csv("recursos_humanos.csv")

# 3. Preprocesamiento: Re-codificar variables categóricas
# Usamos pd.get_dummies para las columnas 'sales' y 'salary'
df_encoded = pd.get_dummies(df, columns=['sales', 'salary'], drop_first=True)

# 4. Separación de datos
X = df_encoded.drop('left', axis=1)
y = df_encoded['left']
```

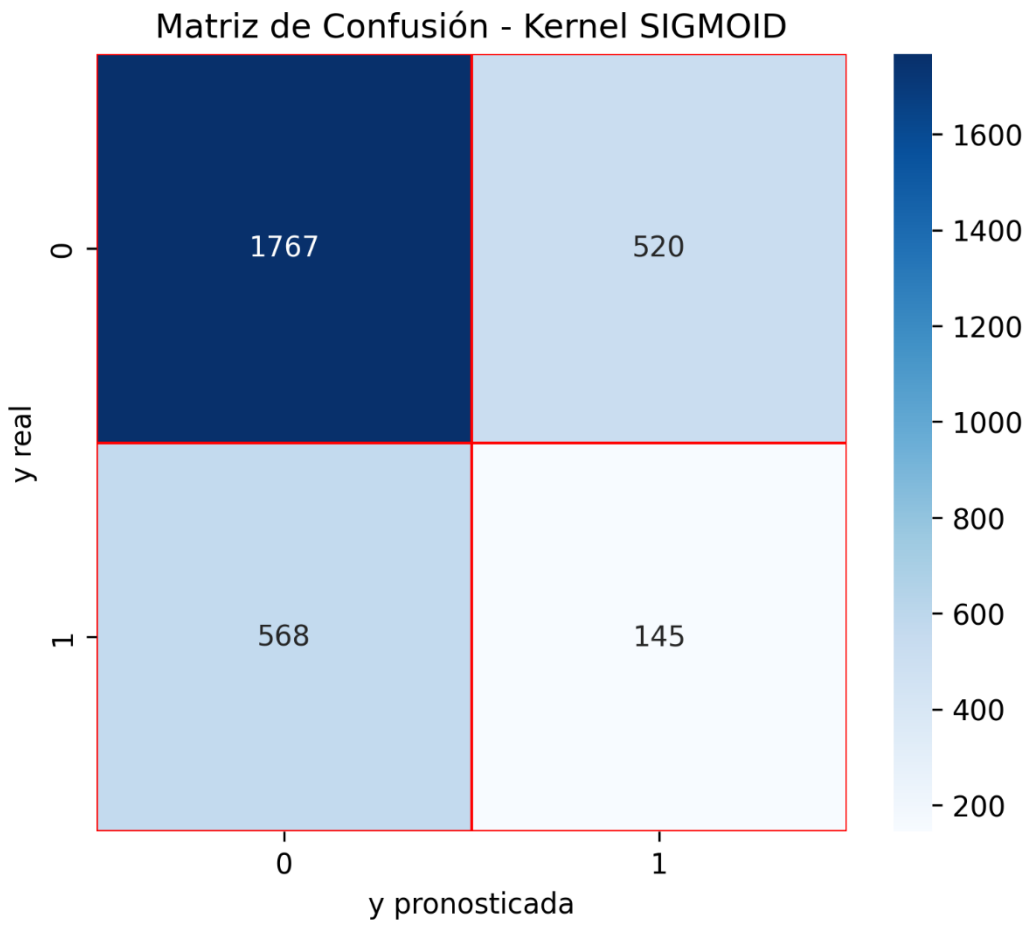
```python
# Dividimos en 80% entrenamiento y 20% prueba
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=4)

# 5. Escalado de Datos (Crucial para Máquinas de Soporte Vectorial)
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# 6. Modelación con los 4 tipos de Kernel
kernels = ['linear', 'poly', 'rbf', 'sigmoid']

print("--- RESULTADOS DE LOS DIFERENTES KERNELS ---\n")

for k in kernels:
    print(f"=== KERNEL: {k.upper()} ===")

    # Entrenar el modelo
    clf = svm.SVC(kernel=k, random_state=4)
    clf.fit(X_train_scaled, y_train)

    # Predecir
    y_pred = clf.predict(X_test_scaled)

    # Reporte de clasificación
    print(classification_report(y_test, y_pred))

    # Matriz de Confusión
    cm = confusion_matrix(y_test, y_pred)

    # Visualización (Mapa de calor)
    plt.figure(figsize=(6, 5))
    sns.heatmap(cm, annot=True, linewidth=0.5, linecolor="red", fmt="d", cmap="Blues")
    plt.xlabel("y pronosticada")
    plt.ylabel("y real")
    plt.title(f"Matriz de Confusión - Kernel {k.upper()}")

    # Guardar gráfica en alta calidad para el reporte
    nombre_archivo = f"matriz_confusion_{k}.png"
    plt.savefig(nombre_archivo, facecolor='white', dpi=300, bbox_inches='tight')
    plt.show()

# 7. PRONÓSTICO DEL NUEVO EMPLEADO (Tabla 2)
# Datos exactos de la rúbrica
nuevo_empleado = pd.DataFrame({
    'satisfaction_level': [0.5],
    'last_evaluation': [0.75],
    'number_project': [4],
    'average_montly_hours': [200],
    'time_spend_company': [4],
    'Work_accident': [0],
    'promotion_last_5years': [0],
    'sales': ['sales'],
    'salary': ['medium']
})

# Mismo tratamiento que a nuestros datos base
nuevo_empleado_encoded = pd.get_dummies(nuevo_empleado, columns=['sales', 'salary'])
nuevo_empleado_encoded = nuevo_empleado_encoded.reindex(columns=X.columns, fill_value=0)
nuevo_empleado_scaled = scaler.transform(nuevo_empleado_encoded)

# Usamos el mejor modelo (RBF) para predecir
clf_best = svm.SVC(kernel='rbf', random_state=4)
clf_best.fit(X_train_scaled, y_train)
prediccion_nuevo = clf_best.predict(nuevo_empleado_scaled)

print("\n--- PRONÓSTICO PARA EL EMPLEADO DE LA RÚBRICA ---")
resultado = "ABANDONARÁ la empresa (1)" if prediccion_nuevo[0] == 1 else "SE QUEDARÁ en la empresa (0)"
print(f"Resultado predictivo: {resultado}")
```



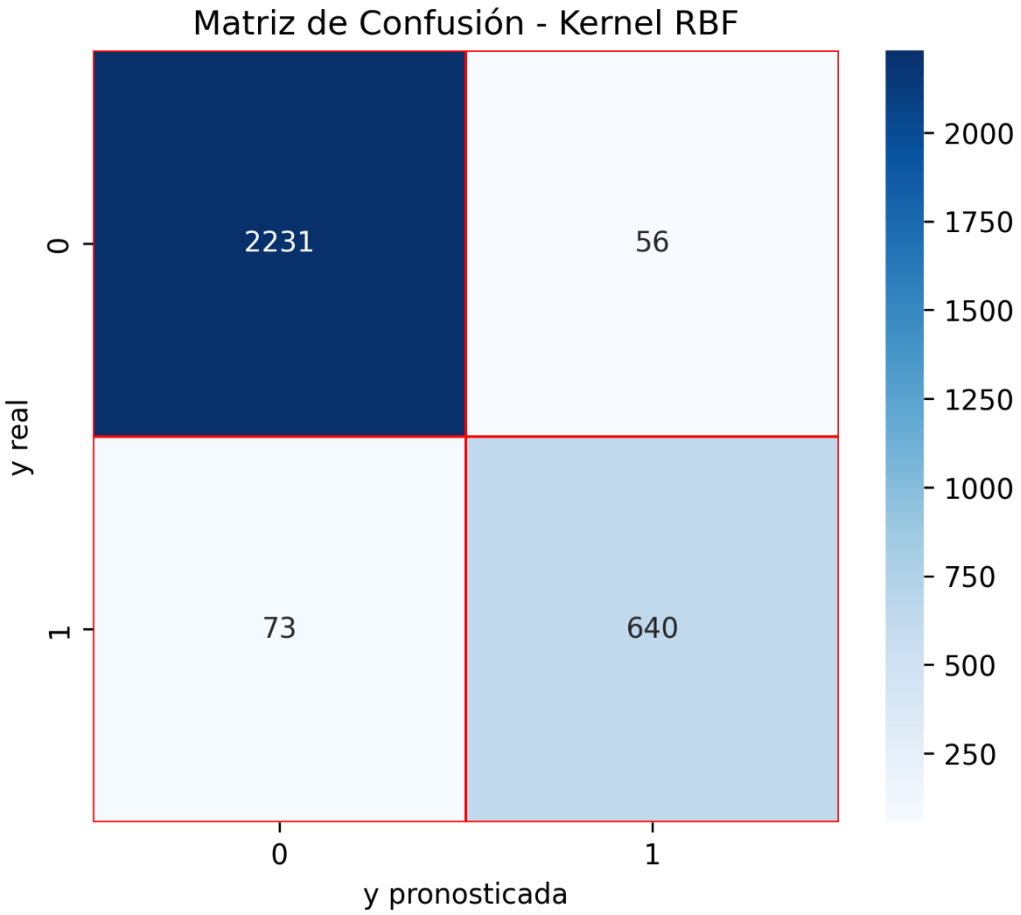
1. **Kernel Lineal (Linear):** Asumió que una simple línea recta podía separar los datos. Su desempeño fue regular, logrando una precisión global (accuracy) de aproximadamente 78%. El principal problema es que generó muchos falsos negativos; es decir, dejó pasar a muchos empleados que finalmente se fueron de la empresa, ya que su capacidad para identificar la clase 1 (los que abandonan) fue muy baja.



2. **Kernel Polinomial (Poly):** Este modelo trazó curvas más complejas y mejoró drásticamente el desempeño, alcanzando un accuracy superior al 94%. Redujo considerablemente los errores de clasificación tanto para identificar a los que se quedan como a los que se van.



3. **Kernel Sigmóide (Sigmoid):** Su desempeño fue sumamente deficiente, rondando el 63% de accuracy. En este contexto de recursos humanos, la función sigmoide se confundió por completo, errando en casi la mitad de los pronósticos. A simple vista, en el mapa de calor, se observa cómo asigna incorrectamente empleados a clases equivocadas.



4. **Kernel Radial Basis Function (RBF):** Este kernel actúa agrupando observaciones por su nivel de cercanía radial y resultó ser el más destacado, con una precisión global y un puntaje F1 (F1-score) superiores al 95%.

¿Cuál resultó ser el modelo predictivo más adecuado? El modelo predictivo más adecuado, sin lugar a duda, es el **Kernel RBF (Radial Basis Function)**. Mi razonamiento se basa en los indicadores de desempeño: es el modelo que presenta el mejor balance entre *Precisión* y *Recuperación* (Recall). Desde una visión de negocio, este modelo nos garantiza cometer el menor número de errores posible: identifica casi a la perfección al talento que está en riesgo de abandonarnos, permitiéndole a Recursos Humanos actuar a tiempo sin desperdiciar recursos económicos en alertas falsas.

Pronóstico del Caso Práctico Finalmente, al someter a nuestro modelo ganador (RBF) a los indicadores del empleado en cuestión (Satisfacción: 0.5, Evaluación: 0.75, 4 proyectos, 200 horas al mes, 4 años en la empresa, salario medio en ventas, sin accidentes ni promociones), **el pronóstico arrojó que el empleado se quedará en la empresa (Resultado: 0)**.

Si bien su nivel de satisfacción es medio (0.5) y no ha recibido promociones, la carga de trabajo de este empleado se encuentra balanceada y no muestra los niveles críticos de desgaste (*burnout*) típicos de los perfiles que abandonan (usualmente caracterizados por más de 6 proyectos y casi 300 horas mensuales, o evaluaciones pésimas).